

Tuples in C# 7 with Examples

In this article, I am going to discuss the **Tuples in C# 7** with Examples. Please read our previous article where we discussed the [Digit Separators in C#](#) with examples. As part of this article, we are going to discuss the following pointers.

1. [Why do we need Tuples in C#?](#)
2. [What are the different ways to return more than one value from a method?](#)
3. [Examples of Tuples Before C# 7.](#)
4. [Understanding the Problems with the Tuples Before C# 7.](#)
5. [How to use Tuples from C# 7.](#)
6. [Tuples in C# with named Parameters](#)
7. [Guidelines to use Tuples](#)

Why do we need Tuples in C#?

If you want to return more than one value from a method then you need to use **Tuples in C#**. And in the programming world, it is a very common thing to return multiple values from a method. Tuples are not completely new in C# 7. In .NET Framework 4.0, a set of **Tuple** classes has been introduced in the **System** namespace. Tuples in C# 7, provide a better mechanism to return multiple values from a method.

What are the different ways to return more than one value from a method in C#?

Following are the different mechanisms available in C# to return multiple values from a method:

1. **Using Custom DataType:** You can return multiple values from a method by using a **custom data type (i.e. class)** as the return type of the method. But sometimes we don't need or don't want to use classes and objects because that's just too much for the given purpose.
2. **Using Ref and Out variable:** You can also return more than one value from the method either by using the **"out"** or **"ref"** parameters. Using **"out"** or **"ref"** parameters is quite difficult to understand and moreover, the **"out"** and **"ref"** parameters will not work with the async methods.
3. **Using dynamic keyword:** You can also return multiple values from a method by using the dynamic keyword as the return type. The **dynamic** keyword was introduced in C# 4. But from a performance point of view, we probably don't want to use dynamic.

As I already told tuples are not new to C# 7. They come with much more improvements in C# 7. So, let us first understand the Tuples which are there before C# 7, and then we will see what improvements they have done with types in C# 7.

Tuples Before C# 7:

In the following example, we are returning two values (integer and double) from the Calculate method using the Tuple class. Here, within the Calculate method, we create an instance of the Tuple class by calling the static Create method of the Tuple class. To the static Create method, we are passing the required integer and double values that we want to return from the method. In the Main method, we are storing the result in a Tuple variable and then access the first value i.e. count using the item1 property and the second value using

the item2 property. The following example code is self-explained, so please go through the comment lines for a better understanding.

```
using System;
using System.Collections.Generic;
namespace TulesDemo
{
    class Program
    {
        static void Main()
        {
            var values = new List<double>() { 10, 20, 30, 40, 50 };
            //Store the Result of Calulate Method in a variable of Tuple type
            Tuple<int, double> t = Calulate(values);
            //Access the First value using Item1 and second value using Item2
            Console.WriteLine($"There are {t.Item1} values and their sum is {t.Item2}");
            Console.ReadKey();
        }
        //Declaring the return type as Tuple<int, double>
        private static Tuple<int, double> Calulate(IEnumerable<double> values)
        {
            int count = 0;
            double sum = 0.0;
            foreach (var value in values)
            {
                count++;
                sum += value;
            }
            //Creating an object of Tuple class by calling the static Create method
            Tuple<int, double> t = Tuple.Create(count, sum);
            //Returning the tuple instance
            return t;
        }
    }
}
```

Problems with the above code:

There are 3 major problems in the above code with Tuple:

1. The **First Problem** is that the Tuples in C# are classes, i.e. reference types. As reference types, the memory is allocated on the Heap Area and Garbage is collected only when they are no longer used. For applications where performance is a major concern, it can be an issue.
2. The **Second Problem** is that the elements in the tuple don't have any names and you can only access them by using the build-in names such as Item1, Item2, Item3, etc. which are not meaningful at all. The **Tuple<T1, T2>** type does not provide any information about what the tuple actually represents which makes it a poor choice in public APIs.
3. The **Third Problem** is that you can use a maximum of 8 properties in a Tuple in C#. If you want to return more than 8 values from a method, then again the last argument of the Tuple must be

another Tuple i.e. Tuple within another Tuple. This makes the syntax more difficult to understand.

How to Overcome the above Problems in C#?

To overcome the above three problems, C# 7 comes with a new feature that is used to improve the support for tuples in C#. With C# 7, now it is possible to declare the tuple as **inline**, which is like an anonymous type, except that they are not limited to the current method. Let's modify the code as shown below to see the use of new improvements of Tuples in C# 7. In your code, if you are getting **Predefined type 'System.ValueTuple'2' is not defined or imported** error, then you need to add the **System.ValueTuple** package from NuGet Package Manager.

```
using System;
using System.Collections.Generic;
namespace TulesDemo
{
    class Program
    {
        static void Main()
        {
            var values = new List<double>() { 10, 20, 30, 40, 50 };
            var result = Calculate(values);
            Console.WriteLine($"There are {result.Item1} values and their sum {result.Item2}");
            Console.ReadKey();
        }

        private static (int, double) Calculate(IEnumerable<double> values)
        {
            int count = 0;
            double sum = 0.0;
            foreach (var value in values)
            {
                count++;
                sum += value;
            }
            return (count, sum);
        }
    }
}
```

As you can see in the above code, we are returning two values i.e. int and double from the Calculate method and then we are accessing the values. This is much better. If you want then you can also give specific names to the tuples returning values.

Tuples in C# 7 with Named Parameters:

With C# 7, now it is possible to provide the tuples parameters with user-defined names. To understand this let's modify the code as shown below. Here we are providing names for the tuple parameters in the Calculate

method as count and sum. Now, in the Main method, you can access these parameters and moreover, you will also get intelligence.

```
using System;
using System.Collections.Generic;
namespace TulesDemo
{
    class Program
    {
        static void Main()
        {
            var values = new List<double>() { 10, 20, 30, 40, 50 };
            var result = Calculate(values);
            Console.WriteLine($"There are {result.count} values and their sum
{result.sum}");
            Console.ReadKey();
        }

        private static (int count, double sum) Calculate(IEnumerable<double> va
        {
            int count = 0;
            double sum = 0.0;
            foreach (var value in values)
            {
                count++;
                sum += value;
            }
            return (count, sum);
        }
    }
}
```

Provide Explicitly Names While Storing the Result:

In the following example, we are providing explicit names to the tuple properties with the Main method. In this case, you don't require to provide the variable name as we can access the properties directly with the provided name.

```
using System;
using System.Collections.Generic;
namespace TulesDemo
{
    class Program
    {
        static void Main()
        {
            var values = new List<double>() { 10, 20, 30, 40, 50 };
            var (countResult, SumResult) = Calculate(values);
            Console.WriteLine($"There are {countResult} values and their sum is
{SumResult}");
        }
    }
}
```

```

        Console.ReadKey();
    }

    private static (int count, double sum) Calculate(IEnumerable<double> values)
    {
        int count = 0;
        double sum = 0.0;
        foreach (var value in values)
        {
            count++;
            sum += value;
        }
        return (count, sum);
    }
}

```

Guidelines to use Tuples:

Basically, one and the most important question that comes to our mind is when to use Tuples and when to use Classes to return more than one value from a method in C#. The answer is it depends on the business requirement. However, there are some guidelines and rules that you need to follow which will guide you to choose between them:

Tuples in C# 7 are values, so they are copied by value, rather than by reference.

Most of the time, this should not be an issue. However, if you are passing around tuples of large structs, this might have an impact on the performance of the application. Ref locals/returns can be used to work around these performance issues that we will discuss in our upcoming articles. As the tuples in C# 7 are values, so modifying a copy will not change the original copy.

Just use common sense

For any situation where you might consider using a tuple: simply ask yourself the question: **“Will a tuple simplify the code here”**. If the answer is **“yes”**, then use a tuple. And that ultimately is the primary consideration over whether to use a tuple or a custom class.

So in simple words, we can say that a Tuple is an ordered sequence of heterogeneous objects. The Tuples in C# are going to be used when a method is going to return more than one value.

//Creating Method to return 2 values using Tuple

```
private static (int count, double sum) Calulate(IEnumerable<double> values)
{
    return (5, 150);
}
```

//Calling the method

```
var (count, sum) = Calulate(values);
```

//Accesing the Tuple Properties

```
Console.WriteLine(count);
```

```
Console.WriteLine(sum);
```

In the next article, I am going to discuss [Splitting Tuples in C# 7](#) with Examples. Here, in this article, I try to explain the **Tuples in C# 7** with Examples. I hope you enjoy this C# Tuples article.